

Arithmetic Operations

Instr.	Attr. 1	Attr. 2	Attr. 3	OP- code	size	pseudo	Instr type	info
ADD	regD	regS	regS	0x01	3			
ADDi	regD	regS	Imm	0x02	4			
ADC	regD	regS	regS	0x03	3			add with carry
SUB	regD	regS	regS	0x04	3			
SUBi	regD	regS	Imm	0x05	4			
SBB	regD	regS	regS	0x06	3			sub with borrow
MUL	regD	regS	regS	0x07	3			multiply lower half
MULH	regD	regS	regS	0x08	3			multiply upper half
MULHu	regD	regS	regS	0x09	3			multiply upper half unsigned
MULHsu	regD	regS	regS	0x0A	3			multiply upper half signed, unsigned
DIV	regD	regS	regS	0x0B	3			
DIVu	regD	regS	regS	0x0C	3			
REM	regD	regS	regS	0x0D	3			remainder
REMu	regD	regS	regS	0x0E	3			
INC	regD	regS		0x01	2	yes		increment by 1
DEC	regD	regS		0x04	2	yes		decrement by 1
INC	regD			0x01	2	yes		
DEC	regD			0x04	2	yes		

Instr.	Attr. 1	Attr. 2	Attr. 3	OP-code	size	pseudo	Instr type	info
CMP	regS	regS		0x0F	2			compare. Sub R1 and R2 and set flags without saving value

Logical Operations

Instr.	Attr. 1	Attr. 2	Attr. 3	OP-code	size	pseudo	Instr type	info
AND	regD	regS	regS	0x10	3			
OR	regD	regS	regS	0x11	3			
XOR	regD	regS	regS	0x12	3			
NOR	regD	regS	regS	0x13	3			
NAND	regD	regS	regS	0x14	3			
NOT	regD	regS		0x15	2			~regS
NEG	regD	regS		0x16	2			~regS + 1
SHL	regD	regS	regS	0x17	3			shift left
SHR	regD	regS	regS	0x18	3			shift right (logical)
SHRa	regD	regS	regS	0x19	3			shift right (arithmetical)
SHRI	regD	regS	regS	0x18	3	yes		
SHLi	regD	regS	Imm	0x1A	4			
SHRi	regD	regS	Imm	0x1B	4			
SHRai	regD	regS	Imm	0x1C	4			
SHRli	regD	regS	Imm	0x1B	4	yes		
ROR	regD	regS		0x1D	2			rotate right

Instr.	Attr. 1	Attr. 2	Attr. 3	OP-code	size	pseudo	Instr type	info
ROL	regD	regS		0x1E	2			rotate left
NOP				0x1F				reserved

Jumps and Branches

(Immediates/Labels in jumps are always an offset)

Every jump is variable. That means you can provide a destination register and a source register.

A normal jump would look like this:

JZ PC, PC, LABEL

Take the value of the PC and add it with the offset. Then put this value into the PC again.

JZ R1, R2, LABEL

Take the value of R2, add it with the offset and put it into R1.

How to jump with any flags:

Use CMP and compare FLAGS reg with your bitmask. Then JZ.

Instr.	Attr. 1	Attr. 2	Attr. 3	OP-code	size	pseudo	Instr type	info
JMP _r	regD	regS	Imm	0x20	4			jump register
JMP	Imm			0x20	4	yes		
JMP	regD	Imm		0x20	4	yes		
JAL _r	regD	regS	Imm	0x21	4			Jump and Link; PC = regS+Offs; regD = PC+4
JAL	Imm			0x21	4	yes		RA = PC+4; PC += offs
JAL	regD	Imm		0x21	4	yes		regD = PC+4; PC += offs
JZ _r	regD	regS	Imm	0x22	4			Jump if Zero
JZ	Imm			0x22	4	yes		
JNZ _r	regD	regS	Imm	0x23	4			Jump if not Zero

Instr.	Attr. 1	Attr. 2	Attr. 3	OP-code	size	pseudo	Instr type	info
JNZ	Imm			0x23	4	yes		
JSr	regD	regS	Imm	0x24	4			Jump if Signed
JS	Imm			0x24	4	yes		
JNSr	regD	regS	Imm	0x25	4			Jump if not Signed
JNS	Imm			0x25	4	yes		
JCr	regD	regS	Imm	0x26	4			Jump if Carry
JC	Imm			0x26	4	yes		
JNCr	regD	regS	Imm	0x27	4			Jump if not Carry
JNC	Imm			0x27	4	yes		
JOr	regD	regS	Imm	0x28	4			Jump if Overflow
JO	Imm			0x28	4	yes		
JNOOr	regD	regS	Imm	-	4	yes		Jump if not Overflow (with bitmasking implemented)
JNO	Imm			-	4	yes		
JPr	regD	regS	Imm	0x29	4			Jump if Parity
JP	Imm			0x29	4	yes		
JNPr	regD	regS	Imm	-	4	yes		Jump if not Parity (with bitmasking implemented)
JNP	Imm			-	4	yes		
BEQ	regS	regS	Imm	0x2A	4			Branch if Equal
BNEQ	regS	regS	Imm	0x2B	4			Branch if not Equal
BGT	regS	regS	Imm	-	4	yes		Branch if Greater Than
BGTs	regS	regS	Imm	-	4	yes		
BLT	regS	regS	Imm	0x2C	4			Branch if Less Than
BLTs	regS	regS	Imm	0x2D	4			

Instr.	Attr. 1	Attr. 2	Attr. 3	OP-code	size	pseudo	Instr type	info
BGEQ	regS	regS	Imm	0x2E	4			Branch if Greater or Equal
BGEQs	regS	regS	Imm	0x2F	4			Branch if Greater or Equal Than
BLEQ	regS	regS	Imm	-	4	yes		Branch if Less or Equal

Memory Operations

Instr.	Attr. 1	Attr. 2	Attr. 3	OP-code	size	pseudo	Instr type	info
SW	regS	regS	Imm	0x30	4			store word
LW	regD	regS	Imm	0x31	4			load word
SB	regS	regS	Imm	0x32	4			store byte
LB	regD	regS	Imm	0x33	4			load byte
WRITE	regS	regS	Imm	0x30	4	yes		
READ	regD	regS	Imm	0x31	4	yes		
WRITEb	regS	regS	Imm	0x32	4	yes		
READb	regD	regS	Imm	0x33	4	yes		
SWi	regS	Imm		0x34	4			Store Immediate
SBi	regS	Imm		0x35	4			
WRITEi	regS	Imm		0x34	4	yes		
WRITEbi	regS	Imm		0x35	4	yes		
MOV	regD	regS		0x36	2			move
MV	regD	regS		0x36	2	yes		
MOVE	regD	regS		0x36	2	yes		

Instr.	Attr. 1	Attr. 2	Attr. 3	OP-code	size	pseudo	Instr type	info
RET	-	-	-	0x36	2	yes		
RETURN	-	-	-	0x36	2	yes		PC = RA
PUSH	regD	regS		0x37	2			push to stack or heap
POP	regD	regS		0x38	2			pop from stack or heap

Other Instructions

Instr.	Attr. 1	Attr. 2	Attr. 3	OP-code	size	pseudo	Instr type	info
NOP	-	-	-	0x00	1			No operation
cFLAGS	-	-	-	0x01	4	yes		clear flags register: (add FLAGS, x0, x0)
NOP				0x39				reserved
NOP				0x3A				reserved
NOP				0x3B				reserved
NOP				0x3C				reserved
NOP				0x3D				reserved
NOP				0x3E				reserved
HALT	-	-	-	0x3F	1			halts the CPU